EIDOS Research Program

Catálogo completo de las versiones estudiadas

Parte I: EIDOS, de NOVA v0.1 a Sparse16. Parte II: proyecto PAOLA, de PAOLA-1 a PAOLA-19, PAOLA-18R, K3-S27, la serie Omega y PAOLA-XFER. Arquitecturas, variantes, baselines, perfiles de escala, experimentos y protocolos, organizados por categorías y explicados uno a uno.

Pablo Sánchez Contento · Investigación independiente · Octubre 2026

Cómo leer este documento. Cada versión tiene una ficha con su estado: **verde** = vigente o mejor resultado; **ámbar** = superada, parcial o sin mejora; **rojo** = invalidada, negativa o rechazada; **gris** = baseline, prueba, bloqueada o planificada. Las cifras proceden de los JSON crudos del repositorio o, cuando se indica, del registro de ejecución conservado (resultado anotado pero cuyo código exacto no está disponible). Los datos de PAOLA (Parte II) proceden de conversaciones de trabajo recuperadas y no tienen código ni JSON en el repositorio.

Contenido

Categoría	Qué incluye	Nº
PARTE I · EIDOS		
1. Linaje de arquitecturas	NOVA v0.1, ISADORA v0.2, SAPPHO v0.3, EIDOS v0.4 (C-M-C-A), EIDOS Sparse16, Sparse12, Sparse8	7
2. Variantes internas de EIDOS v0.4	EIDOS Core, Wide Memory, Dual Anchor, Pulse	4
3. Baselines de comparación	Transformer completo, Conv Striped, Recurrent Striped (y conv_striped de NOVA)	3
4. Familia de escala (perfiles)	Smoke, Glyph 10M, Verse 30M, Epic 100M, Mythos 180M, perfil 427M, perfil 1.093B	7
5. Experimentos de componentes	Scan clamp vs. composición afín, Pulse stride 4, fusión de parámetros, token-shift, FlexAttention, checkpointing simétrico, medición en proceso aislado	7
6. Experimentos de calidad	0,1 · 0,5 · 2,0 tokens/parámetro	3
PARTE II · PAOLA		
7. Objetivo, entorno y criterios	Metas, hardware, gates de promoción, mapa de la investigación	—
8. Primeras arquitecturas	PAOLA-1, 2, 3, 3D, 4, 5A, 5B, 5C, 5D, 5E, 5F	11
9. Fase de velocidad	PAOLA-6, 7, 7 G4096, 8 TimeSlice16, 9 CBH-MoE4, 10 Titan, 11	7
10. Familia TITAN	PAOLA-12 a 19, PAOLA-18R y la corrección numérica BF16/Adafactor	10
11. Validación y series derivadas	K3-S27, serie Omega Ω01–Ω10 (Ω03 MZero, Ω04 Dilate124…), campaña de 9 entrenamientos, PAOLA-XFER	9
12. Cierre y runtime	PLE, skips, MTP (6 y 1 token), FFN-Q3mix, lecciones y datos que faltan	4
13. Resumen global	Tablas comparativas de EIDOS y PAOLA, conclusión y siguientes pasos	—

La idea que recorre todo el proyecto es **falsacionista**: cada versión se diseñó para intentar demostrar que la anterior estaba equivocada. Por eso la mayoría de las fichas acaban en rojo o ámbar. Eso no es un fracaso del documento: es justamente lo que permitió llegar a una arquitectura plausible (EIDOS Sparse16) sin engañarse por el camino.

Glosario rápido

Término	Significado
Throughput (tok/s)	Tokens de entrenamiento procesados por segundo. Mide velocidad, no inteligencia.
bits/token	Calidad de predicción: cuántos bits necesita el modelo para codificar cada token. Menos es mejor.
tokens/parámetro (tpp)	Cuántos tokens de entrenamiento ve el modelo por cada parámetro. Los modelos bien entrenados suelen estar en ~20 tpp; 0,1–2,0 es muy poco entrenamiento.
Parameter matching	Igualar el número de parámetros entre los modelos comparados (margen ≤0,5%) para que ninguno gane por ser más grande.
Macrociclo	Patrón de tipos de capa que se repite a lo largo de la profundidad del modelo.
Scan	Algoritmo que calcula de forma paralela una recurrencia (estado que se arrastra token a token).
Eager / compilado	PyTorch ejecutando operación a operación (eager) o con el grafo optimizado por torch.compile.
Test sellado	Conjunto de evaluación que solo se usa una vez, al final, con todas las decisiones congeladas.

1. Linaje de arquitecturas

Son las versiones principales del proyecto, en orden cronológico. Cada una nace de los problemas detectados en la anterior. La pregunta común es siempre la misma: *¿puede un modelo de lenguaje entrenado desde cero en una GPU de consumo (RTX 5070, 12 GB) competir con un Transformer del mismo tamaño gastando menos cómputo?*

```
NOVA v0.1 → ISADORA v0.2 → SAPPHO v0.3 → EIDOS v0.4 → EIDOS Sparse16
                                                        ├─ Sparse12 (probada)
                                                        └─ Sparse8  (planificada)
```

NOVA v0.1 (NOVA-Lattice)	**INVALIDADA**
Idea	Introducir **ABIM** (Adaptive Budget Innovation Memory): una memoria que solo escribe la *innovación*, es decir, lo que el estado anterior no era capaz de predecir, con un presupuesto de escritura adaptativo.
Escala y datos	Modelos de menos de un millón de parámetros, a nivel de byte (sin tokenizer BPE).
Lo que parecía	Un **cruce de escalado**: a partir de cierto tamaño NOVA mejoraba más rápido que el baseline convolucional (conv_striped).
Qué falló	1) NOVA recibió una rejilla de learning rate más amplia que el baseline; la variación de NOVA solo por cambiar el LR era 2–4 veces mayor que la ventaja arquitectónica reportada. 2) La variante ganadora de las ablations tenía ~3% más parámetros y una sola semilla. 3) El scan por chunks tenía un bug numérico silencioso (ver categoría 5). 4) Los resultados byte-level no tienen por qué trasladarse a BPE de 32k, donde la redundancia que explota la memoria se comprime en el tokenizer.
Veredicto	El cruce de escalado **no quedó demostrado**; podía deberse a la selección de hiperparámetros, no a la arquitectura.
Qué se heredó	El concepto de memoria de innovación (escribir la sorpresa, no la copia), que sobrevive hasta Pulse Memory en EIDOS.

ISADORA v0.2	**NEGATIVA**
Idea	No cambiar la tesis, sino **endurecer el método**: scan corregido, tokenizer BPE, matching de parámetros estricto y un protocolo de benchmark serio.
Cambios técnicos	Scan sustituido por composición afín estable (sin clamp); tests de causalidad, backward, caché y chunks grandes; pipeline BPE.
Resultado	En una confirmación corta perdió frente al Transformer y el informe **no declaró victoria**.
Auditoría posterior	Una revisión externa detectó que los proxies BPE apenas habían aprendido: vocabulario de 1.024 piezas, baseline uniforme de 10 bits/token y resultados de 16–27 bits/token (peor que adivinar al azar). 19 de 19 selecciones de LR caían en el borde superior de la rejilla (la rejilla era demasiado baja). Solo se usaron entre 1.920 y 6.400 tokens por corrida (~1–3% del corpus proxy). Dos matrices incumplían la tolerancia de parámetros (spread de 0,347% y 0,523%).
Veredicto	Evidencia insuficiente para cualquier conclusión de superioridad. Se reinterpretó también la ablation *no_surprise*: el LR sí era igual, pero toda su evidencia estaba dentro de matrices sin aprendizaje útil.
Qué se heredó	Barreras automáticas: rechazo si no se supera el baseline uniforme, si el LR toca el borde, si hay pocos tokens o si el matching de parámetros falla.

SAPPHO v0.3	DESCARTADA
Idea	Mantener la **atención como columna vertebral** (como un Transformer normal) y añadir la memoria predictiva como una rama residual lateral (*sidecar*) de ancho completo, con compuerta casi cerrada al inicio y LayerScale para no romper la optimización temprana.
Prueba real	Benchmark en RTX 5070 real, PyTorch 2.13.0+cu130, BF16, ~10M parámetros, diferencia de parámetros 0,0496%.
Resultados (JSON crudo)	**Eager:** 4.131 tok/s vs. 11.714 del Transformer → 0,353x. **Compilado:** 12.141 vs. 25.171 tok/s → 0,482x. Compilar aceleró SAPPHO 2,94x, pero el Transformer seguía siendo 2,07x más rápido. VRAM compilada: 0,238 GiB vs. 0,220 GiB.
Diagnóstico	La topología duplicaba trabajo: cada bloque calculaba atención *y* memoria completa. El perfilado mostró que la capa de memoria de ancho completo era más cara que la propia atención, y algunos caminos del benchmark repetían la proyección al vocabulario.
Veredicto	Estructuralmente lenta. Se abandonó la topología y también el nombre (se decidió no usar nombres de personas).

EIDOS v0.4 — macrociclo C-M-C-A	SUPERADA
Nombre	**Efficient Innovation-Driven Ordered State**. Eidos = forma o esencia.
Idea	Pasar de «atención + memoria en cada bloque» a **capas especializadas**: cada capa hace una sola cosa. Macrociclo de cuatro capas: C = convolución causal, M = memoria Pulse, C, A = ancla de atención global.
Mezclador C	Convolución causal depthwise con compuerta: proyección a 2d, conv temporal por canal (kernel 7), SiLU × sigmoide, proyección de salida. Coste lineal en la longitud.
Pulse Memory (M)	Estado de rango bajo (d/8 en los perfiles principales); escribe la innovación respecto a la predicción anterior, normalizada por su RMS; presupuesto de escritura condicionado por la sorpresa; tres escalas de tiempo aprendibles y ordenadas (decays en [0,85–0,97], [0,965–0,995], [0,995–0,9997]) compartidas entre canales; routers de lectura y escritura independientes; scan por chunks con cumsum renormalizado, sin clamp, chunk máximo 128 para evitar graph breaks de torch.compile; oráculo secuencial float32 para verificarlo; estado constante durante la generación.
Ancla A	Atención causal global con grouped-query attention (GQA), RoPE y normalización de q/k.
Herramientas	train.py con test sellado y mínimo de 20 tokens/parámetro; benchmark_gpu.py; build_matrix.py; select_lrs.py (bloquea si el mejor LR toca un borde); evaluate_gate.py (puerta final); deduplicación exacta antes del split; tokenizer BPE de 32.768 piezas desde cero.
Resultado	Diagnóstico en CPU (secuencia 512): EIDOS Core 1,290x el throughput del Transformer. Es un diagnóstico de implementación, no evidencia CUDA. Es la última instantánea con **código completo** en el repositorio (SHA-256 784872f1…7fc9).
Veredicto	Corrige el problema de SAPPHO sobre el papel; fue la base sobre la que se construyó Sparse16.

EIDOS Sparse16 (rama post-v0.4)	MEJOR RESULTADO
Idea	Reducir todavía más la frecuencia de memoria y atención: en cada bloque de 16 capas, 14 son locales, 1 es memoria y 1 es atención global.
Macrociclo	L L L L L L L M L L L L L L L A — en un modelo de 48 capas: 42 mezcladores locales, 3 memorias y 3 anclas globales.
Novedades	Mezclador local con **token-shift aprendido** ($s_t = \alpha \odot x_{t-1}$, una ruta recurrente mínima), controles de Pulse fusionados, FlexAttention compilado, checkpointing simétrico y medición de memoria en procesos aislados.
Resultado clave	~1.093B parámetros, contexto 4096, BF16, Adafactor: **5.229 tok/s vs. 4.049** del Transformer emparejado (**1,291x**); memoria reservada 5,084 vs. 5,279 GiB (**−3,7%**); asignada 4,591 vs. 4,604 GiB.
Calidad	Ventaja de 0,166 bits/token a 0,1 tpp, que se reduce a 0,0172 a 0,5 tpp; a 2,0 tpp sin cierre (ver categoría 6).
Limitación	El árbol de código exacto y los JSON crudos de esta rama **no están en el archivo conservado**. El resultado está anotado en el registro de ejecución, pero no es reproducible desde el repositorio.
Veredicto	La dirección de ingeniería más prometedora del proyecto. Superioridad de calidad no demostrada.

EIDOS Sparse12	SIN MEJORA
Idea	Comprobar si la brecha de calidad de Sparse16 se debía a tener muy pocas capas globales. Se aumentó la frecuencia de memoria y atención (ciclo de 12 capas en lugar de 16).
Resultado	No mejoró de forma material la validación en el modelo pequeño.
Conclusión	Añadir más atención global **no basta** para explicar la diferencia de calidad. Se mantuvo Sparse16 como topología de referencia por ser más barata.

EIDOS Sparse8	PLANIFICADA
Idea	Ciclo todavía más denso (memoria y atención cada 8 capas).
Estado	Incluida en la fase 2 del roadmap: comparar Sparse8, Sparse12 y Sparse16 con parámetros, tokens, optimizador, búsqueda de LR y tiempo de reloj igualados. Aún no medida.

2. Variantes internas de EIDOS v0.4

Dentro de v0.4 se implementaron cuatro variantes que se diferencian en el orden de las capas del macrociclo o en el rango de la memoria. Sirven para responder a preguntas concretas: ¿importa dónde va la memoria? ¿cuánta memoria hace falta? ¿ayuda tener la atención antes? Todas se emparejan en parámetros ajustando el ancho de la FFN (matching analítico con candidatos alineados a 8).

Variante	Macrociclo	Rango memoria	Parámetros	FFN	tok/s CPU	vs. Transformer
EIDOS Core	C-M-C-A	32 (d/8)	11.049.963	640	1.754	**1,290x**
EIDOS Wide Memory	C-M-C-A	64 (2×)	11.058.379	632	1.432	1,053x
EIDOS Dual Anchor	C-A-M-A	32	11.048.171	640	1.600	1,177x
EIDOS Pulse	M-C-M-A	32	11.049.430	672	1.421	1,046x
Transformer (ref.)	A-A-A-A	—	11.045.120	608	1.359	1,000x

Datos: `results/v0.4/cpu_diagnostic.json`, 8 hilos, secuencia 512, mediana de 5 pasos. Son diagnósticos de implementación en CPU, no evidencia en GPU.

EIDOS Core	**VIGENTE**
Qué es	La variante de referencia: dos mezcladores convolucionales, una memoria y una atención por cada cuatro capas, memoria de rango d/8.
Pregunta	¿Elimina la topología especializada la penalización de SAPPHO?
Resultado	La más rápida en el diagnóstico CPU (1,290x). Es la candidata que se usa como objetivo de parámetros para emparejar el resto de modelos en train.py.

EIDOS Wide Memory	**PRUEBA**
Qué es	Igual que Core, pero con el doble de rango en la memoria (64 en Glyph); la FFN se estrecha para mantener el mismo número de parámetros.
Pregunta	¿Mejora la calidad si la memoria tiene más capacidad?
Coste observado	Baja a 1,053x en CPU: una memoria más ancha cuesta velocidad. Su valor depende de si compensa en calidad, algo que requiere entrenamiento en GPU.

EIDOS Dual Anchor	**PRUEBA**
Qué es	Macrociclo C-A-M-A: dos anclas de atención por ciclo y una sola convolución.
Pregunta	¿Hace falta más comunicación global para no perder calidad?
Coste observado	1,177x en CPU: más atención, menos velocidad. Anticipa el experimento Sparse12 de la rama posterior, que también subió la frecuencia global sin ganar calidad.

EIDOS Pulse	**PRUEBA**
Qué es	Macrociclo M-C-M-A: dos memorias por ciclo.
Pregunta	¿Aporta más memoria predictiva en profundidad?
Coste observado	1,046x en CPU, la más lenta de las cuatro. La memoria es la pieza cara; de ahí que la rama Sparse16 tomara justo el camino contrario (memoria muy escasa).

3. Baselines de comparación

Un resultado solo significa algo si el rival está bien construido. Todos los baselines comparten embeddings, normalización, FFN SwiGLU y la misma lógica de matching de parámetros que EIDOS.

Transformer completo (transformer_full)	BASELINE
Patrón	A en todas las capas: atención causal global GQA + FFN SwiGLU.
Papel	El rival principal. Toda puerta de promoción exige superarlo en calidad en test sellado y alcanzar al menos 1,05x su throughput en la RTX 5070.
Lección	Su backend de atención decide la comparación: un Transformer con backend ineficiente produjo la falsa ventaja de 23x (categoría 4, perfil 427M).

Conv Striped (conv_striped)	BASELINE
Patrón	C-C-C-A: tres convoluciones causales y una atención, sin memoria.
Papel	Aísla el efecto de la memoria: si EIDOS no supera a Conv Striped, la memoria Pulse no está aportando nada. Es también el baseline contra el que NOVA v0.1 pareció ganar.
Dato	1,276x en el diagnóstico CPU, prácticamente igual que EIDOS Core (1,290x): la velocidad viene sobre todo de sustituir atención por convolución, así que la memoria debe justificarse en calidad.

Recurrent Striped (recurrent_striped)	BASELINE
Patrón	R-C-R-A, donde R es un *GatedStateMixer*: recurrencia diagonal de bajo rango con las mismas tres escalas de tiempo, pero **sin innovación predictiva** (escribe tanh(valor), no la sorpresa).
Papel	Control directo de la hipótesis central: separa «tener un estado recurrente» de «escribir solo lo que no se predijo». Si empata con EIDOS, la idea de innovación no aporta.

4. Familia de escala (perfiles)

Los nombres Glyph, Verse, Epic y Mythos son **niveles de evidencia**, no etiquetas comerciales: cada uno solo se entrena si el anterior ha superado su puerta. Todos usan vocabulario BPE de 32.768 piezas (salvo Smoke), kernel 7, 3 bandas de memoria, chunk 64 y macrociclo C-M-C-A.

Perfil	Parámetros	dim	Capas	Cabezas / KV	FFN	Rango mem.	Contexto	Checkp.	MTP
Smoke	41.279	32	4	4 / 2	64	12	64	no	0
Glyph	11,05M	256	4	4 / 2	640	32	2048	no	0
Verse	26,22M	384	8	6 / 2	1152	48	2048	sí	0
Epic	98,48M	640	12	10 / 2	2816	80	4096	sí	0,05
Mythos	173,81M	768	20	12 / 4	2560	96	4096	sí	0,05

MTP = peso de la cabeza auxiliar de predicción multi-token. En comparaciones de throughput se desactiva o se iguala.

Smoke	**PRUEBA**
Función	Modelo diminuto (vocabulario 128) para comprobar que todo el pipeline funciona de principio a fin.
Resultado	Se entrenó desde pesos aleatorios y generó checkpoint final; el override de sub-entrenamiento queda marcado explícitamente para que nadie lo lea como evidencia de calidad. Lo ejecuta la suite de tests en CPU.

Glyph (~11M)	**VIGENTE**
Función	Instrumento: primera comparación real en GPU y calibración de learning rate.
Estado	En Glyph todos los gaps de parámetros quedan por debajo de 0,08%; el Transformer queda a 0,044% del objetivo. Es el perfil del diagnóstico CPU de la categoría 2.

Verse (~26M)	**BLOQUEADA**
Función	Repetición con más semillas y calidad por tiempo de reloj.
Estado	No debe entrenarse hasta analizar el benchmark compilado de Glyph en la RTX 5070.

Epic (~98M)	**BLOQUEADA**
Función	Escala, código y contexto largo (4096), con checkpointing y MTP.
Estado	Bloqueado tras la puerta de Verse.

Mythos (~174M)	**BLOQUEADA**
Función	El modelo grande de la familia v0.4.
Puerta	Solo `evaluate_gate.py` puede desbloquearlo, y únicamente si una misma variante: supera el baseline uniforme; usa ≥20 tokens/parámetro; tiene ≥3 semillas; mantiene gap de parámetros ≤0,5%; vence al mejor baseline en test sellado por ≥0,5%; y alcanza ≥1,05x el throughput del Transformer en la RTX 5070.

Perfil 427M · contexto 2048	RECHAZADA
Qué pasó	Una comparación inicial mostró una ventaja de ~23x en velocidad y una diferencia de memoria enorme.
Por qué se rechazó	El Transformer probablemente cayó fuera de un backend de atención eficiente y el checkpointing no se aplicó por igual a los dos modelos.
Lección	Una arquitectura no puede «ganar» porque el baseline esté roto. La paridad de backend pasó a formar parte del protocolo.

Perfil 1.093B · contexto 4096 (Sparse16)	MEJOR RESULTADO
Configuración	1.093.209.171 parámetros, BF16, Adafactor, checkpointing simétrico, procesos aislados.
Resultado	EIDOS 5.229 tok/s vs. Transformer 4.049 tok/s → **1,291x**; reservada −3,7%.
Frontera de VRAM	Sin checkpointing, el modelo «cabía» casi en el límite de 12 GB, pero Windows WDDM empezó a paginar y el throughput cayó a ~265 tok/s. Con checkpointing simétrico subió por encima de 5.000 tok/s. «Cabe en VRAM» no equivale a «es entrenable en la práctica».

5. Experimentos de componentes

Además de las arquitecturas completas se estudiaron versiones alternativas de piezas concretas. Varias de estas pruebas explican más del resultado final que la propia arquitectura.

Scan con clamp_min(1e-12) vs. composición afín	**INVALIDADA**
Versión original	El scan diagonal por chunks dividía por productos acumulados de decays y aplicaba `clamp_min(1e-12)` para «estabilizar».
Problema	El clamp cambiaba la matemática de la recurrencia. Error ~4e-7 con chunk 32, pero **mayor que 3** con chunks 64 y 128, sin NaNs ni avisos: un bug silencioso.
Versión corregida	ISADORA: composición afín estable. EIDOS v0.4: cumsum renormalizado sin clamp, chunk máximo 128 y un oráculo secuencial float32 que lo verifica en tests con seis tamaños de chunk.

Pulse Memory con stride 4	**SIN MEJORA**
Idea	Actualizar la memoria solo cada 4 tokens para abaratarla.
Resultado	Mejoró el componente aislado, pero no el modelo completo: los costes de agrupar, repetir y particionar el grafo del compilador se comieron casi toda la ganancia local.

Fusión de parámetros (optimizador)	**VIGENTE**
Problema	Gran parte de la penalización de EIDOS venía de tener muchos tensores de parámetros pequeños en Adafactor, no de los FLOPs.
Solución	Fusionar las proyecciones de control (lectura, escritura, presupuesto) y los parámetros de estado. Redujo el tiempo de optimizador de forma sustancial.
Lección	Un benchmark de arquitectura puede estar dominado por cómo se organizan los parámetros.

Token-shift aprendido	**VIGENTE**
Qué es	Una ruta directa del token anterior: $s_t = \alpha \odot x_{t-1}$, con α diagonal aprendido.
Por qué	Da una propagación causal mínima sin recurrencia matricial completa; en generación solo necesita el estado del token anterior en cada capa local.
Pendiente	Ablation sin token-shift (fase 2 del roadmap).

FlexAttention compilado	**VIGENTE**
Contexto	El PyTorch disponible en Windows no exponía la ruta FlashAttention esperada para la configuración GQA probada.
Solución	Usar FlexAttention compilado para las anclas globales, y aplicar el mismo criterio de backend eficiente al Transformer para mantener la paridad.

Checkpointing simétrico	**VIGENTE**
Qué es	Recalcular activaciones en el backward en lugar de guardarlas, aplicado por igual a EIDOS y al Transformer.
Por qué importa	Fue la diferencia entre ~265 tok/s (paginación WDDM) y >5.000 tok/s a 1,1B, y su ausencia asimétrica invalidó el resultado de 23x.

Medición en proceso aislado	**VIGENTE**
Qué es	Medir cada modelo en un proceso nuevo, y en orden invertido, para evitar que la caché del allocator, la temperatura de la GPU o el orden de ejecución contaminen las cifras de memoria y velocidad.

6. Experimentos de calidad

La velocidad es solo la mitad de la pregunta. Para calidad se compararon EIDOS y Transformer con el mismo tokenizer, corpus, parámetros, presupuesto de tokens y semillas emparejadas, midiendo bits/token en validación (LR 1e-2 en los presupuestos cortos).

Presupuesto	Semillas	EIDOS	Transformer	Diferencia	Estado
0,1 tokens/parámetro	3 emparejadas	—	—	−0,166 bits/token (media)	Prometedor, muy sub-entrenado
0,5 tokens/parámetro	3 emparejadas	11,5330	11,5502	−0,0172 bits/token	Ventaja estrecha
2,0 tokens/parámetro	1 extensión	—	—	Sin evaluación final común	Sin cerrar

0,1 tokens/parámetro — PARCIAL

Lectura	EIDOS ganó de forma estable en las tres semillas. Pero 0,1 tpp es ~200 veces menos que un entrenamiento razonable: a ese nivel, ganar puede significar solo «aprende más rápido al principio».

0,5 tokens/parámetro — PARCIAL

Lectura	Con cinco veces más entrenamiento, la ventaja se redujo casi diez veces, a 0,0172 bits/token. Señal real pero pequeña y en descenso.

2,0 tokens/parámetro — SIN MEJORA

Lectura	No se completó la comparación final sobre un conjunto fijo común. Durante la extensión, un checkpoint del Transformer obtuvo un punto de validación momentáneo mejor. Por eso el proyecto **no afirma ninguna victoria de calidad**.

PARTE II · Proyecto PAOLA

7. PAOLA: objetivo, entorno y criterios

PAOLA fue el programa de exploración de arquitecturas que se desarrolló en paralelo y a continuación de la línea EIDOS (referencias de julio de 2026, trabajo intensivo del 11 al 13 de agosto de 2026 y cierre posterior). Abarca las familias **PAOLA-1 a PAOLA-19**, varias subvariantes, la versión corregida **PAOLA-18R**, el experimento de validación **K3-S27**, la serie **Ω (Omega)** vinculada a EIDOS, el protocolo de transferencia **PAOLA-XFER** y optimizaciones de runtime posteriores.

Importante: son iteraciones de arquitectura y experimentos, no 19 modelos de lenguaje entrenados por completo.

Procedencia de los datos. Todo lo de esta parte se ha reconstruido a partir de las conversaciones de trabajo. El repositorio **no** contiene el código fuente, los checkpoints ni los JSON de benchmark de PAOLA. Las cifras son las que se registraron en su momento; cuando falta un dato, se dice así y no se rellena. Los resultados de distintas versiones proceden de configuraciones de prueba distintas y no siempre son comparables entre sí.

Objetivo

- Al menos un **25 % más rápida** que un Transformer equivalente (≥1,25× de throughput).
- Igual o menos VRAM (≤1,00×).
- Mantener o mejorar la calidad lingüística.
- Escalar bien a contextos de 4.096 tokens.
- Entrenable en hardware doméstico con PyTorch.
- Incorporar mecanismos de memoria alternativos a la atención tradicional.

No se buscaba solo una pérdida baja, sino una mejor relación entre calidad, parámetros y coste computacional.

Entorno de pruebas

Componente	Configuración
GPU	NVIDIA RTX 5070, 12 GB de VRAM
CPU / RAM	AMD Ryzen 7 5800X · 32 GB DDR4
Sistema y framework	Windows · PyTorch 2.13 + CUDA 13.0
Precisión	BF16; después parámetros maestros FP32 con autocast BF16 (ver corrección numérica)
Compilación	torch.compile en modo fullgraph
Contextos	512 y 4.096 tokens
Tamaños	~150M, ~427M y ~27M parámetros
Control	Transformer con GQA, mismos parámetros, datos y precisión

Criterios de promoción (gates)

Métrica	Requisito
Throughput relativo	≥1,25× el Transformer
VRAM relativa	≤1,00× el Transformer
Comparaciones	Mismos parámetros, datos y precisión
Consistencia	Normalmente 7 pares de pruebas (p. ej. «7/7» = los 7 pares superan el umbral)
Calidad	BPT (bits por token, menos es mejor) y pruebas independientes

Métrica	Requisito
Validez	Causalidad, gradientes y reproducibilidad

Mapa de la investigación PAOLA

```
Fase 1  PAOLA-1 … 5F            memorias alternativas (~150M)
Fase 2  PAOLA-6 … 11            velocidad a cualquier coste
Fase 3  TITAN: PAOLA-12 … 19    ~427M, ctx 4096, (7C+M+7C+A)×2
          └─ PAOLA-18R          corrección numérica FP32
Fase 4  K3-S27 · Omega Ω01–Ω10 · PAOLA-XFER   validación y derivadas
Fase 5  PLE · skips · MTP · FFN-Q3mix        optimización de runtime
```

8. Primeras arquitecturas PAOLA (1 a 5F)

Primera fase: probar mecanismos de memoria distintos de la atención (DeltaNet, RLS, memoria lineal, fast-weights) a escala de ~150M parámetros.

Versión	Diseño o mecanismo	Resultado recuperado
PAOLA-1	Gated DeltaNet-2 + atención local, bloques recurrentes	0,636–0,642× velocidad; mejor BPT inicial
PAOLA-2	Memoria delta matricial, actualización rank-1	Compilación excesiva; sin benchmark completo válido
PAOLA-3	Memoria RLS + atención GQA	~0,849×, lastrada por operaciones pequeñas
PAOLA-3D	Variación de PAOLA-3	Inferior al control; faltan detalles
PAOLA-4	Memoria lineal sin solve ni bucles costosos	Hasta 0,982× con contexto 4096
PAOLA-5A	GDN2 de 8 capas	~149,95M parámetros; propuesta MODIFY
PAOLA-5B	LOOM	Diseño propuesto, sin mediciones
PAOLA-5C	MOSAIC-LOOM	Diseño propuesto, sin mediciones
PAOLA-5D	MOSAIC-3R	1,234553×; VRAM 0,9426×
PAOLA-5E	MOSAIC-3R Lite	Hasta 1,2626×, pero perdió calidad
PAOLA-5F	Local-Preserve	Propuesta correctiva, sin resultados concluyentes

PAOLA-1 · GDN2 + atención local — DESCARTADA

Diseño	Cinco bloques **Gated DeltaNet-2** (GDN2, una memoria recurrente con regla delta y compuertas) y una atención local, con bloques compartidos y ejecución recurrente. La implementación evaluada tenía ~172,68M parámetros y 14 repeticiones.
Calidad	BPT inicial **9,094682** frente a **9,250398** del Transformer (−0,156): ligera ventaja.
Velocidad / VRAM	0,636–0,642× el throughput del control (≈36 % más lenta). VRAM ~1,868 GiB vs. 1,83 GiB.
Veredicto	Primera señal interesante de calidad, pero no pasó el filtro de rendimiento.

PAOLA-2 · memoria delta matricial — INCOMPLETA

Diseño	Memoria delta en forma de matriz con actualización de rango 1 por token.
Resultado	El tiempo de compilación fue excesivo y no llegó a obtenerse un benchmark completo válido.

PAOLA-3 · memoria RLS + GQA — DESCARTADA

Diseño	10 capas (7 de memoria RLS —mínimos cuadrados recursivos— y 3 de GQA), ~150M parámetros, contexto 512. Chunks de 32 tokens comprimidos en 4 prototipos; estado de memoria M = 64×24 y matriz auxiliar P = 24×24.
Intención	Actualizar una memoria temporal de forma eficiente y recuperar información sin atención completa en todas las capas.
Resultado	39.494 tok/s con chunk 128 (~0,848× del control). Con chunk 32, solo **0,1135×**.
Problema	Implementación: muchas operaciones pequeñas, actualizaciones de matrices y sobrecarga de lanzamiento en GPU.

PAOLA-3D — DESCARTADA

Diseño	Variación de PAOLA-3.
Resultado	Inferior al control. No se han recuperado más detalles.

PAOLA-4 · memoria lineal	SUPERADA
Diseño	PAOLA-3 sin las operaciones `solve` ni algunos bucles costosos.
Resultado	Contexto 512, batch 4: 43.635 tok/s (~0,8406×) con 1,174× VRAM. Contexto 4096: hasta **0,982×** el throughput del Transformer.
Veredicto	Buena mejora de ingeniería, pero lejos del objetivo de 1,25×.

PAOLA-5 · concepto general	PROPUESTA
Diseño	Memoria **Block Fast-Weight**, bloques anchos y poco profundos, dos atenciones GQA locales y estados resumidos por macro-bloque. De aquí salen LOOM y MOSAIC, pensadas para comprimir y combinar información.

PAOLA-5A · GDN2 de 8 capas	PROPUESTA
Diseño	Versión GDN2 de 8 capas con ~149,95M parámetros.
Estado	Quedó como propuesta con decisión MODIFY (modificar antes de medir).

PAOLA-5B · LOOM	PROPUESTA
Estado	Diseño propuesto; no se han recuperado mediciones.

PAOLA-5C · MOSAIC-LOOM	PROPUESTA
Diseño	Jerarquía de cuatro tokens por estado, núcleo global trabajando con una cuarta parte de los tokens y cabeza léxica de doble factor.
Estado	Diseño propuesto; sin mediciones recuperadas.

PAOLA-5D · MOSAIC-3R	PARCIAL
Resultado	**1,234553×** de throughput y **0,9426×** de VRAM.
Lectura	Primera versión claramente más rápida y ligera que el control, aunque se quedó justo por debajo del umbral de 1,25×.

PAOLA-5E · MOSAIC-3R Lite	DESCARTADA
Resultado	61.201 tok/s frente a 48.473 del Transformer con batch 4 → **1,2626×**; VRAM ~0,931×.
Problema	El BPT empeoró en **+0,4892**.
Veredicto	Pasaba el gate de velocidad pero se descartó por calidad.

PAOLA-5F · Local-Preserve	PROPUESTA
Idea	Propuesta correctiva para recuperar la calidad perdida (preservar el procesamiento local).
Estado	Sin resultados concluyentes recuperados.

9. PAOLA-6 a 11: velocidad sin perder calidad

Fase agresiva de pruebas. Algunas arquitecturas mejoraron mucho el rendimiento, pero la calidad se resistía. 1,00× es el Transformer de control; cada fila es una configuración de prueba distinta.

Versión	Velocidad relativa	VRAM relativa	Resultado
PAOLA-6	~1,00×	Mayor que el control	Descartada
PAOLA-7	1,091×	Mayor que el control	Descartada
PAOLA-7 G4096	Sin cifra fiable	—	Cobertura insuficiente (75–76 %)
PAOLA-8 TimeSlice16	**1,3427×**	**0,8988×**	Rápida, peor calidad
PAOLA-9 CBH-MoE4	Operador 0,813×	—	Gate de eficiencia fallido
PAOLA-10 Titan	1,2597×	1,01429×	Falla por VRAM
PAOLA-11	Sin registro	—	No verificable

PAOLA-6 — DESCARTADA

Resultado	~1,00× de velocidad (sin ganancia) y más VRAM que el control.

PAOLA-7 — DESCARTADA

Resultado	1,091× de velocidad pero más VRAM que el control: no cumple ninguno de los dos gates.

PAOLA-7 G4096 — INCOMPLETA

Qué es	Prueba de PAOLA-7 con contexto 4096.
Resultado	Sin cifra fiable: la cobertura de la prueba se quedó en el 75–76 %, insuficiente para dar un número.

PAOLA-8 TimeSlice16 — DESCARTADA

Resultado	Una de las más rápidas de toda la investigación: **1,3427×** de velocidad y **0,8988×** de VRAM (~10 % menos memoria).
Calidad	Déficit de +0,1905 BPT a 4,19M tokens que **creció** a +0,2662 BPT a 16,78M tokens.
Veredicto	Un déficit que aumenta con el entrenamiento es mala señal: se descartó.

PAOLA-9 CBH-MoE4 — DESCARTADA

Diseño	Operador de mezcla de expertos (MoE de 4 expertos) con dos implementaciones.
Resultado	Denso: 2,104 ms. Packed: 1,710 ms. Relación 0,813.
Veredicto	El objetivo exigía una relación ≤0,50. Gate de eficiencia fallido.

PAOLA-10 Titan — DESCARTADA

Resultado	1,2597× de velocidad (pasa el umbral) pero 1,01429× de VRAM.
Veredicto	Falla por VRAM, aunque por poco. Su nombre anticipa la familia TITAN siguiente.

PAOLA-11 — INCOMPLETA

Estado	No se ha recuperado ningún registro cuantitativo. No verificable.

10. Familia TITAN: PAOLA-12 a PAOLA-19

Cambio de escala: de ~150M a ~**427M parámetros**, contexto **4096** y **32 capas**. El diseño usa sobre todo convoluciones causales, algunas capas de memoria y muy pocas de atención global.

```
Embeddings (vocabulario 32.768, dimensión 1.024)
  ┌── macrociclo ×2 ──────────────────────────┐
  │ 7×C (conv. causal) → 1×M (Pulse Memory)   │
  │ 7×C (conv. causal) → 1×A (atención GQA)   │
  └───────────────────────────────────────────┘
Normalización → cabeza de salida → logits
Total: 28 C + 2 M + 2 A = 32 capas (cada bloque con su FFN)
```

Es el mismo patrón de 16 capas que EIDOS Sparse16 (`L×7 M L×7 A`, Parte I), repetido dos veces.

Componente	Valor	Componente	Valor
Capas	32	Convolución	kernel 7
d_model	1024	Memoria (rank)	96–128 según variante
Cabezas Q / KV	16 / 4	Bandas · chunk · stride	3 · 64 · 8
FFN	~3024–3072	Contexto	4096

Versión	Throughput	VRAM	Mejor BPT recuperado
PAOLA-12 NoShift	1,287270×	0,977675×	15,096299
PAOLA-13 MT96	1,290946×	0,977306×	15,103855
PAOLA-14 PartialShift25	Sin cifra exacta	—	15,110675
PAOLA-15	Pasó el gate de sistema	—	No superó el gate de calidad
PAOLA-16	—	~1,0127×	Descartada
PAOLA-17	—	~1,0127×	Descartada
PAOLA-18 MeanLast	**1,304396×**	0,977289×	15,1095642
PAOLA-18R corregida	1,292330×	0,983740×	**8,2832108***
PAOLA-19	Propuestas sucesivas	—	Sin resultado final

* PAOLA-18R usó otro protocolo numérico (tras la corrección FP32). Su BPT **no** es comparable con los de calibración de PAOLA-12 a 18 (~15,1), que además estaban afectados por el problema de BF16/Adafactor.

PAOLA-12 NoShift — SUPERADA

Diseño	TITAN sin token-shift en los mezcladores (según su nombre).
Resultado	1,287270× · VRAM 0,977675× · BPT 15,096299 (el mejor de la calibración original).
Nota	Su calidad quedó en entredicho por el problema numérico descubierto después.

PAOLA-13 MT96 — SUPERADA

Diseño	TITAN con memoria de rango 96 (MT96, según su nombre).
Resultado	1,290946× · VRAM 0,977306× · BPT 15,103855.

PAOLA-14 PartialShift25 — SUPERADA

Diseño	Token-shift parcial (PartialShift25: por el nombre, aplicado al 25 % de los canales).
Resultado	Sin cifra exacta de throughput recuperada · BPT 15,110675.

PAOLA-15	SIN MEJORA
Resultado	Superó el gate de sistema (velocidad/VRAM) pero no el de calidad. Métricas detalladas no recuperadas.

PAOLA-16 y PAOLA-17	DESCARTADA
Resultado	Ambas con VRAM ~1,0127×, por encima del límite de 1,00×. Descartadas.

PAOLA-18 MeanLast	SUPERADA
Diseño	Variante MeanLast de TITAN.
Resultado	**Mejor velocidad TITAN** recuperada: 1,304396× con 0,977289× de VRAM · BPT 15,1095642.
Nota	Su calibración original de calidad no vale como prueba definitiva (problema BF16/Adafactor). Fue la base de PAOLA-18R.

Corrección numérica BF16 / Adafactor	VIGENTE
Qué se encontró	Las pruebas de calidad iniciales de PAOLA-12 a PAOLA-18 estaban afectadas por el uso de BF16 con Adafactor. Magnitud de actualización registrada: Transformer **0,001722**, mezcladores PAOLA **0,000152**, unas **11,3 veces** menos.
Consecuencia	Los mezcladores PAOLA apenas se actualizaban: las diferencias de calidad podían deberse a una actualización deficiente de parámetros y no a la arquitectura.
Corrección	Parámetros maestros en FP32 con autocast BF16, además de otras verificaciones.
Importancia	El descubrimiento metodológico más importante de PAOLA. Dio lugar a PAOLA-18R.

PAOLA-18R (corregida)	MEJOR RESULTADO
Qué es	PAOLA-18 reevaluada con el protocolo numérico corregido.
Sistema	Throughput **1,29233×** · VRAM **0,98374×** · **7/7** pares por encima de 1,25×.
Calidad	BPT **8,28321** frente a **8,61164** del Transformer: **−0,328428 BPT** a favor de PAOLA-18R.
Condiciones	8.552.448 tokens de entrenamiento, learning rate 0,003, semilla 1337.
Limitación	Solo ~0,02 tokens por parámetro, una semilla y sin evaluación independiente. No demuestra que la ventaja se mantenga tras cientos de millones de tokens.
Veredicto	El mejor resultado combinado preliminar de PAOLA: una señal experimental, no una victoria sobre el Transformer.

PAOLA-19	INCOMPLETA
Estado	Hubo propuestas sucesivas, pero no hay ningún resultado final confirmado.

11. Validación y series derivadas

K3-S27 · prueba de calidad a pequeña escala	PENDIENTE
Para qué	Comprobar si la ventaja de PAOLA-18R se mantiene con más entrenamiento sin tener que entrenar 427M parámetros a ciegas. Mide la *evolución* de la calidad, no solo el punto inicial.
Configuración	~27M parámetros · vocabulario 8.192 · dimensión 256 · 32 capas · cabezas Q/KV 4/1 · rank de memoria 24 · contexto 4096 · 3 semillas.
Presupuesto	5 tokens/parámetro iniciales, ampliables hasta 15. LR elegido entre 0,0003 · 0,001 · 0,003 · 0,01.
Requisitos	Entrenamiento y validación separados, tokenizador entrenado solo con el conjunto de entrenamiento, mismo presupuesto para todos los modelos.
Estado	No se ha recuperado un resultado final completo y validado.

Serie Omega (Ω01–Ω10)

Tras PAOLA-18R se investigaron arquitecturas complementarias, algunas dentro de la línea EIDOS. Entre los candidatos de la serie estaban: **A12, Dilate, DynV, DynC4, 1G2S, KDA2, GDN2, Raven2, PGDN2 y Mamba3-2**. No se ha recuperado la correspondencia exacta entre cada candidato y su número Ω, salvo en los casos de abajo.

Métrica	Ω03 MZero	Ω04 Dilate124	Transformer
Throughput	1,2992×	1,2871×	1,00×
VRAM	0,9944×	0,9834×	1,00×
BPT inicial	**8,70959**	8,72314	9,05419
Diferencia BPT	**−0,34460**	−0,33105	—
Gate de sistema	7/7	7/7	—

Ω03 MZero	PROMETEDORA
Diseño	30 capas convolucionales y 2 de atención (sin capas de memoria).
Resultado	1,2992× · VRAM 0,9944× · BPT 8,70959 vs. 9,05419 (−0,3446). Gate de sistema 7/7.
Lectura	La mejor candidata Omega preliminar por BPT. Pendiente de varias semillas y entrenamiento prolongado. Si «MZero» significa cero capas de memoria, como sugiere su composición, plantea si la memoria era necesaria.

Ω04 Dilate124	PROMETEDORA
Diseño	Convoluciones con distintas dilataciones (el nombre indica 1, 2 y 4), memoria y 2 capas de atención.
Resultado	1,2871× · VRAM 0,9834× · BPT 8,72314 (−0,33105). Gate de sistema 7/7.

Ω02	DESCARTADA
Resultado	1,1245× de throughput, por debajo del umbral.

Ω05	SIN MEJORA
Resultado	1,2583×, pero no superó todos los criterios.

Ω08	PENDIENTE
Resultado	1,2809×, sin validación definitiva.

Resto de la serie (Ω01, Ω06, Ω07, Ω09, Ω10)	INCOMPLETA
Estado	Varias de las demás pruebas resultaron inválidas o incompletas. No se han recuperado cifras individuales.

Campaña Omega de 9 entrenamientos	PENDIENTE
Plan (13 ago.)	3 arquitecturas × 3 semillas, 27M parámetros, 5 tokens/parámetro, entrenamientos emparejados.
Estado	Consta Ω04 con semilla 3407 como quinto entrenamiento. No hay veredicto final de la campaña.

PAOLA-XFER · transferir un Transformer a PAOLA	PROPUESTA
Idea	Partir de un Transformer ya entrenado en vez de entrenar PAOLA desde cero: copiar embeddings, FFN, normalizaciones y componentes GQA compatibles, y conservar solo dos capas de atención elegidas por su importancia.
Protocolo	1) Alineación local: 32M tokens. 2) Conversión progresiva: 64M. 3) Destilación global de conocimiento (KD): 96M. 4) Entrenamiento causal adicional: 64M. **Total: 256M tokens.**
Criterio de éxito	Perder como máximo +0,05 BPT respecto al profesor, conservar ≥95 % de las capacidades cortas y ≥90 % de las pruebas de recuperación e inducción.
Estado	Protocolo propuesto; no hay registro de una transferencia completada.

12. Cierre de PAOLA y optimizaciones posteriores

Entre el 12 y el 13 de agosto la conclusión cambió: había arquitecturas que pasaban el gate de velocidad y VRAM, pero no evidencia sólida de calidad madura ni de superioridad general. La investigación pasó a optimizaciones de ejecución y arquitectura.

PLE de bajo rango, skips y PAOLA-PLE	**SIN MEJORA**
Qué se probó	PLE de bajo rango, conexiones skip y su integración en PAOLA (PAOLA-PLE).
Resultado	Los experimentos PLE no funcionaron como se esperaba. No hay cifras recuperadas de los skips.

MTP con 6 tokens de borrador	**DESCARTADA**
Qué se probó	Predicción multi-token (MTP) generando 6 tokens de borrador por paso.
Resultado	No funcionó como se esperaba.

MTP con 1 token (n=1)	**VIGENTE**
Resultado	Prueba: 50/50 vs. 45/50 del control · velocidad 47,3 vs. 45,8 tok/s · VRAM 8,46 vs. 8,63 GiB.
Lectura	Se conservó como línea de optimización de **runtime**, no como victoria arquitectónica de PAOLA.

FFN-Q3mix	**PROPUESTA**
Qué es	Cuantización selectiva de las FFN sobre Gemma 4 E4B, con objetivos estrictos de tamaño, pérdida, divergencia KL y velocidad.
Estado	Sin validación final.

Qué aprendimos con PAOLA

Pregunta	Conclusión
¿Arquitecturas más rápidas que el Transformer?	Sí, en determinados benchmarks
¿Menos VRAM?	Sí, en varias versiones
¿Mejor BPT inicial en alguna?	Sí: PAOLA-1, PAOLA-18R y Omega
¿Mejor calidad tras entrenamiento largo?	No de manera concluyente
¿Escalabilidad a modelos grandes?	No demostrada
¿Errores de metodología?	Sí, sobre todo BF16/Adafactor
¿Nueva arquitectura superior confirmada?	No
¿Valor técnico e investigador?	Sí: diseños, pruebas, errores detectados y protocolos

Datos de PAOLA que faltan

- Código fuente original completo, checkpoints y JSON individuales de cada benchmark.
- Métricas de PAOLA-11 y PAOLA-15, y resultado final de PAOLA-19.
- Resultado final de K3-S27 y de la campaña Omega de 9 entrenamientos.
- Correspondencia completa entre los candidatos Omega y sus números Ω.

13. Resumen global

Parte I · EIDOS

Versión	Categoría	Qué probaba	Resultado	Estado
NOVA v0.1	Linaje	Memoria de innovación ABIM	Cruce de escalado no demostrado	Invalidada
ISADORA v0.2	Linaje	Método endurecido + BPE	Perdió; proxies sin aprendizaje útil	Negativa
SAPPHO v0.3	Linaje	Atención + memoria lateral	0,353x eager / 0,482x compilado	Descartada
EIDOS v0.4 C-M-C-A	Linaje	Capas especializadas	1,290x en CPU; código completo	Superada
EIDOS Sparse16	Linaje	Memoria y atención 1/16	1,291x a 1,093B; −3,7% VRAM	Mejor resultado
EIDOS Sparse12	Linaje	Más capas globales	Sin mejora de calidad	Sin mejora
EIDOS Sparse8	Linaje	Aún más global	No medida	Planificada
EIDOS Core	Variante v0.4	Referencia C-M-C-A	1,290x CPU	Vigente
EIDOS Wide Memory	Variante v0.4	Rango de memoria 2×	1,053x CPU	Prueba
EIDOS Dual Anchor	Variante v0.4	Dos atenciones (C-A-M-A)	1,177x CPU	Prueba
EIDOS Pulse	Variante v0.4	Dos memorias (M-C-M-A)	1,046x CPU	Prueba
Transformer	Baseline	Rival principal	Referencia 1,000x	Baseline
Conv Striped	Baseline	Sin memoria (C-C-C-A)	1,276x CPU	Baseline
Recurrent Striped	Baseline	Estado sin innovación	Control de la hipótesis	Baseline
Smoke	Escala	Pipeline extremo a extremo	Funciona; no es evidencia	Prueba
Glyph 11M	Escala	Instrumento GPU	Gaps <0,08%	Vigente
Verse / Epic / Mythos	Escala	Escalado por puertas	No entrenados	Bloqueadas
427M ctx 2048	Escala	Contexto largo	Falso 23x	Rechazada
1.093B ctx 4096	Escala	Ingeniería a gran escala	5.229 vs. 4.049 tok/s	Mejor resultado
Scan con clamp	Componente	Estabilidad numérica	Error >3 silencioso	Invalidada
Pulse stride 4	Componente	Memoria más barata	No mejora el modelo completo	Sin mejora
Fusión de parámetros	Componente	Coste del optimizador	Gran reducción de tiempo	Vigente
Token-shift	Componente	Ruta causal mínima	Integrado en Sparse16	Vigente
FlexAttention	Componente	Backend de atención	Paridad en Windows	Vigente
Checkpointing simétrico	Componente	Memoria y paridad	265 → >5.000 tok/s	Vigente
Proceso aislado	Componente	Medición limpia	Protocolo estándar	Vigente
Calidad 0,1 / 0,5 / 2,0 tpp	Calidad	Bits/token	−0,166 → −0,0172 → sin cierre	Parcial

Parte II · PAOLA

Versión	Fase	Throughput	VRAM	Calidad / nota	Estado
PAOLA-1	Inicial	0,636–0,642×	~1,02×	BPT −0,156	Descartada
PAOLA-2	Inicial	—	—	Sin benchmark válido	Incompleta
PAOLA-3 / 3D	Inicial	~0,849×	—	Ops. pequeñas	Descartada
PAOLA-4	Inicial	≤0,982×	1,174× (ctx 512)	—	Superada
PAOLA-5A/5B/5C/5F	Inicial	—	—	Diseños propuestos	Propuesta
PAOLA-5D MOSAIC-3R	Inicial	1,2346×	0,9426×	—	Parcial
PAOLA-5E 3R Lite	Inicial	1,2626×	~0,931×	BPT +0,4892	Descartada
PAOLA-6 / 7	Velocidad	~1,00× / 1,091×	>1×	—	Descartada
PAOLA-7 G4096	Velocidad	—	—	Cobertura 75–76 %	Incompleta
PAOLA-8 TimeSlice16	Velocidad	**1,3427×**	**0,8988×**	BPT +0,2662	Descartada
PAOLA-9 CBH-MoE4	Velocidad	Op. 0,813	—	Exigía ≤0,50	Descartada
PAOLA-10 Titan	Velocidad	1,2597×	1,01429×	Falla VRAM	Descartada
PAOLA-11	Velocidad	—	—	Sin registro	Incompleta
PAOLA-12 NoShift	TITAN	1,2873×	0,9777×	BPT 15,0963*	Superada
PAOLA-13 MT96	TITAN	1,2909×	0,9773×	BPT 15,1039*	Superada
PAOLA-14 PartialShift25	TITAN	—	—	BPT 15,1107*	Superada
PAOLA-15	TITAN	Pasa	—	Falla calidad	Sin mejora
PAOLA-16 / 17	TITAN	—	~1,0127×	Falla VRAM	Descartada
PAOLA-18 MeanLast	TITAN	**1,3044×**	0,9773×	BPT 15,1096*	Superada
PAOLA-18R	TITAN	1,2923×	0,9837×	**BPT −0,3284**	Mejor resultado
PAOLA-19	TITAN	—	—	Sin resultado final	Incompleta
K3-S27	Validación	—	—	Sin resultado final	Pendiente
Ω03 MZero	Omega	1,2992×	0,9944×	**BPT −0,3446**	Prometedora
Ω04 Dilate124	Omega	1,2871×	0,9834×	BPT −0,3311	Prometedora
Ω02 / Ω05 / Ω08	Omega	1,1245× / 1,2583× / 1,2809×	—	—	Desc. / sin mejora / pend.
PAOLA-XFER	Transferencia	—	—	256M tokens previstos	Propuesta
MTP n=1	Runtime	47,3 vs. 45,8 tok/s	8,46 vs. 8,63 GiB	50/50 vs. 45/50	Vigente
PLE · MTP 6 · FFN-Q3mix	Runtime	—	—	No funcionó / sin validar	Sin mejora / propuesta

* BPT de calibración original, afectado por el problema BF16/Adafactor. No comparable con el de PAOLA-18R ni con Omega, que usan otro protocolo.

Conclusión

El proyecto refutó su propio diseño inicial (memoria como añadido lateral) y llegó a una arquitectura híbrida escasa más plausible. Lo que **sí** está respaldado: SAPPHO era estructuralmente más lento que un Transformer emparejado (JSON crudo); la computación híbrida escasa es una dirección de ingeniería creíble; y el registro conservado contiene un resultado de 1,291x a 1,1B que merece reproducirse.

Lo que **no** está respaldado: superioridad general de calidad frente a Transformers, rendimiento de estado del arte, ventaja independiente del hardware, novedad de cada componente por separado o preparación para uso comercial.

PAOLA llegó a la misma frontera por otro camino: encontró varias arquitecturas que pasan el gate de velocidad y VRAM (PAOLA-8, PAOLA-18, PAOLA-18R, Ω03, Ω04) y algunas señales iniciales de calidad, pero **no demostró** una arquitectura superior al Transformer tras un entrenamiento maduro. Su aportación más valiosa fue metodológica: la corrección BF16/Adafactor enseñó que una comparación de calidad puede estar decidida por la precisión numérica y no por la arquitectura. Lo que merece reproducción rigurosa es PAOLA-18R y Omega (Ω03, Ω04).

Siguientes pasos

- Recuperar el código exacto de Sparse16 y sus JSON crudos, con hashes publicados.
- Cerrar la comparación a 2,0 tpp en un conjunto fijo grande, con tres semillas y test sellado.
- Ablations decisivas: sin Pulse Memory, sin token-shift, Sparse8 / 12 / 16, rango y escalas de memoria.
- Capacidades: recuperación en contexto largo, recall asociativo, copia selectiva, código, SQL, razonamiento verificable, español e inglés.
- Reproducción en Linux CUDA y en al menos otra generación de GPU, con energía y tiempo hasta calidad.
- PAOLA: completar K3-S27 y la campaña Omega de 9 entrenamientos; reproducir PAOLA-18R, Ω03 y Ω04 con varias semillas y entrenamiento prolongado.